UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT

Data Structure and Algorithm

Laboratory Activity No. 11

Implementation of Graphs

Submitted by:
Gabijan, Rhovic M.

Instructor:
Engr. Maria Rizette H. Sayo

October 18, 2025

I. Objectives

Introduction

A graph is a visual representation of a collection of things where some object pairs are linked together. Vertices are the points used to depict the interconnected items, while edges are the connections between them. In this course, we go into great detail on the many words and functions related to graphs.

An undirected graph, or simply a graph, is a set of points with lines connecting some of the points. The points are called nodes or vertices, and the lines are called edges.

A graph can be easily presented using the python dictionary data types. We represent the vertices as the keys of the dictionary and the connection between the vertices also called edges as the values in the dictionary.

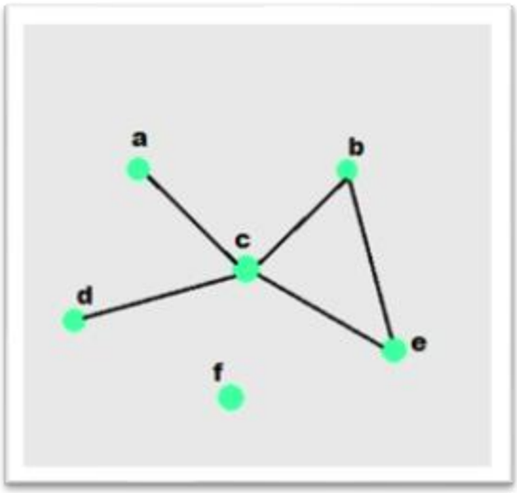

Figure 1. Sample graph with vertices and edges

This laboratory activity aims to implement the principles and techniques in:

- To introduce the Non-linear data structure – Graphs
- To implement graphs using Python programming language
- To apply the concepts of Breadth First Search and Depth First Search

II. Methods

A. Copy and run the Python source codes.
B. If there is an algorithm error/s, debug the source codes.
C. Save these source codes to your GitHub.

```
from collections import deque

class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        """Add an edge between u and v"""
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []

        self.graph[u].append(v)
        self.graph[v].append(u)  # For undirected graph

    def bfs(self, start):
        """Breadth-First Search traversal"""
        visited = set()
        queue = deque([start])
        result = []

        while queue:
            vertex = queue.popleft()
            if vertex not in visited:
                visited.add(vertex)
                result.append(vertex)
                # Add all unvisited neighbors
                for neighbor in self.graph.get(vertex, []):
                    if neighbor not in visited:
                        queue.append(neighbor)
        return result

    def dfs(self, start):
        """Depth-First Search traversal"""
        visited = set()
        result = []

        def dfs_util(vertex):
            visited.add(vertex)
            result.append(vertex)
            for neighbor in self.graph.get(vertex, []):
                if neighbor not in visited:
                    dfs_util(neighbor)

        dfs_util(start)
        return result

    def display(self):
        """Display the graph"""
        for vertex in self.graph:
            print(f"{vertex}: {self.graph[vertex]}")

# Example usage
if __name__ == "__main__":
    # Create a graph
```

```
g = Graph()

# Add edges
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)
g.add_edge(3, 4)

# Display the graph
print("Graph structure:")
g.display()

# Traversal examples
print(f"\nBFS starting from 0: {g.bfs(0)}")
print(f"DFS starting from 0: {g.dfs(0)}")

# Add more edges and show
g.add_edge(4, 5)
g.add_edge(1, 4)

print(f"\nAfter adding more edges:")
print(f"BFS starting from 0: {g.bfs(0)}")
print(f"DFS starting from 0: {g.dfs(0)}")
```

Questions:

1. What will be the output of the following codes?
2. Explain the key differences between the BFS and DFS implementations in the provided graph code. Discuss their data structures, traversal patterns, and time complexity. How does the recursive nature of DFS contrast with the iterative approach of BFS, and what are the potential advantages and disadvantages of each implementation strategy?
3. The provided graph implementation uses an adjacency list representation with a dictionary. Compare this approach with alternative representations like adjacency matrices or edge lists.
4. The graph in the code is implemented as undirected. Analyze the implications of this design choice on the add_edge method and the overall graph structure. How would you modify the code to support directed graphs? Discuss the changes needed in edge addition, traversal algorithms, and how these modifications would affect the graph's behavior and use cases.
5. Choose two real-world problems that can be modeled using graphs and explain how you would use the provided graph implementation to solve them. What extensions or modifications would be necessary to make the code suitable for these applications? Discuss how the BFS and DFS algorithms would be particularly useful in solving these problems and what additional algorithms you might need to implement.

III. Results

```
PS C:\Users\Administrator\Documents\VS CODE> &
Graph structure:
0: [1, 2]
1: [0, 2]
2: [0, 1, 3]
3: [2, 4]
4: [3]

BFS starting from 0: [0, 1, 2, 3, 4]
DFS starting from 0: [0, 1, 2, 3, 4]

After adding more edges:
BFS starting from 0: [0, 1, 2, 4, 3, 5]
DFS starting from 0: [0, 1, 2, 3, 4, 5]
PS C:\Users\Administrator\Documents\VS CODE>
```

1.

2. The difference between BFS and DFS implementations lie in their data structures, traversal patterns, and execution strategies. BFS (breadth-First Search) uses an approach supported by a queue, allowing it to explore nodes level by level, starting from the source and visiting all neighboring vertices before moving deeper. On the other hand, DFS (Depth-First Search) relies on recursion and the system's call stack to explore as far a down path as possible before tracking to previously visited nodes.

3. The graph implementation uses an adjacency list, storing only existing edges, which makes it memory-efficient for sparse graphs ($O(V + E)$) and allows fast access to neighbors for traversal. In contrast, an adjacency matrix uses $O(V^2)$ space, enabling automatic edge lookup but wasting memory in sparse graphs. An edge list stores all edges directly ($O(E)$), which is simple but inefficient for neighbor queries. While matrices and edge lists are appropriate for thick graphs or edge-focused tasks, adjacency lists generally offer a decent trade-off between memory consumption and traversal efficiency.

4. The graph is undirected, so each edge connects nodes in both directions, reflected in the add_edge method by updating both adjacency lists. This simplifies traversal and models mutual relationships. To support directed graphs, edges would be added in only one direction, changing graph behavior to allow one-way connections. BFS and DFS would work without major changes, but the graph could now represent structures like social media follows, web links, or dependency networks.

5. Graphs can model real-world problems like city maps and social networks. In city maps, intersections are nodes and roads are edges, with BFS useful for shortest paths and weighted edges enabling Dijkstra's algorithm. In social networks, people are nodes and friendships or follows are edges; BFS finds

degrees of separation, while DFS detects communities. To use the provided graph code, one might add support for weighted or directed edges and implement additional algorithms for analysis, making it suitable for practical applications.

IV. Conclusion

In conclusion, this graph implementation highlights key concepts in graph theory, including traversal methods, data structures, and different ways to represent graphs. BFS and DFS show two distinct ways of exploring a graph—BFS moves level by level using a queue, while DFS dives deep into paths using recursion. Using an adjacency list provides an efficient way to store sparse graphs, and although the current design is undirected, it can easily be adapted for directed graphs to model one-way relationships. Real-world examples, such as finding shortest paths in city maps or analyzing connections in social networks, demonstrate how these concepts can be applied, and show the value of extending the basic implementation with features like weighted edges or specialized algorithms for practical problem-solving.

References

[1] Co Arthur O.. “University of Caloocan City Computer Engineering Department Honor Code,” UCC-CpE Departmental Policies, 2020.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., Cambridge, MA, USA: MIT Press, 2022. Available: https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/ MIT Press+3MIT Press+3Google Books+3

[3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Boston, MA, USA: Addison-Wesley, 2011. Available: https://algs4.cs.princeton.edu/ algs4.cs.princeton.edu+2Free Computer Books+2

[4] D. B. West, *Introduction to Graph Theory*, 2nd ed., Upper Saddle River, NJ, USA: Prentice Hall, 2001. Available: https://webhomes.maths.ed.ac.uk/~v1ranick/papers/wilsongraph.pdf dwest.web.illinois.edu+2School of Mathematics+2

[5] S. Even (ed. G. Even), *Graph Algorithms*, 2nd ed., Cambridge, NY, USA: Cambridge University Press, 2012. Available: https://www.cambridge.org/core/books/graph-algorithms/8B295BD0845A174FFE6B2CD6D4B2C63F Cambridge University Press & Assessment+2cris.tau.ac.il+2